

END-TO-END MACHINE LEARNING & WEB DEPLOYMENT

CREDIT CARD APPROVAL PREDICTION SYSTEM

A complete underwriting automation system detailing demographic exploration,
feature engineering pipelines, multi-algorithm training,
and dynamic web panel serving.

COMPREHENSIVE TECHNICAL MANUAL & SOURCE CODE

Includes: eda.py | model.py | app.py | templates & styles

Date: June 2026

Status: Verified & Operational

1. Executive Summary & Underwriting Scenarios

Banks and financial institutions receive thousands of credit card applications every day. Auditing each application manually is both time-consuming and highly prone to human error, making it extremely inefficient to scale. This project automates the credit card approval decision using machine learning. By training a predictive model on historical applicant demographic and financial history, the system evaluates applicant parameters to determine creditworthiness instantly.

Three target application scenarios are supported:

- Scenario 1: Automated Credit Card Application Screening

A bank credit analyst enters a new applicant's financial profile (income type, employment duration, credit history) into the dashboard. The model returns an instant approval/rejection prediction, allowing the analyst to prioritize high-value applications and speed up processing.

- Scenario 2: High-Risk Applicant Compliance Review

A compliance officer uses the platform to batch-screen applicants. The pipeline processes multi-class monthly status records and converts payment history into binary labels, allowing the model to clearly classify high-risk applicants as ineligible for a new card.

- Scenario 3: Customer Self-Service Credit Card Eligibility Check

A prospective customer enters their personal and financial details before submitting a formal application, instantly predicting their approval likelihood. This helps the customer understand their eligibility, reducing unnecessary rejections and protecting credit scores.

2. Exploratory Data Analysis (eda.py)

Exploratory Data Analysis (EDA) is vital for understanding demographics and credit risk proportions. The script 'eda.py' merges application records with credit status histories, computes the maximum monthly delinquency tier for each customer, and generates high-resolution charts saved inside 'static/images/':

- approval_distribution.png: Visualizes the proportion of Approved (88%) vs. Rejected (12%) applicants.
- income_distribution.png: Kernel Density Plot representing annual income distributions (truncated at \$500k).
- education_vs_approval.png: Count plot showing approval outcomes across academic qualifications.
- income_type_vs_approval.png: Count plot representing decisions across employment sectors.

Source Code: eda.py

```
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg') # Non-interactive backend to run on server
import matplotlib.pyplot as plt
import seaborn as sns

print("Starting EDA generation...")

# Load datasets
app = pd.read_csv('application_record.csv')
credit = pd.read_csv('credit_record.csv')

# Preprocess target using Option B
# Good (0) = 'C', 'X', '0'
# Bad (1) = '1', '2', '3', '4', '5'
status_map = {'C': 0, 'X': 0, '0': 0, '1': 1, '2': 1, '3': 1, '4': 1, '5': 1}
credit['STATUS_NUM'] = credit['STATUS'].map(status_map)

# Group by client and get maximum status (meaning if they ever had a 30+ day late payment, they are classified as Bad/1)
target = credit.groupby('ID')['STATUS_NUM'].max().reset_index()
target.rename(columns={'STATUS_NUM': 'label'}, inplace=True)

# Merge
df = app.merge(target, on='ID', how='inner')
df_clean = df.drop_duplicates(subset='ID').copy()
df_clean['OCCUPATION_TYPE'] = df_clean['OCCUPATION_TYPE'].fillna('Unknown')

# Set styling
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.family'] = 'sans-serif'

# 1. Count plot of Approval Status
plt.figure()
ax = sns.countplot(x='label', data=df_clean, hue='label', palette=['#10b981', '#ef4444'], legend=False)
plt.title('Distribution of Credit Card Approvals (Option B)', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Application Status', fontsize=12)
plt.ylabel('Count', fontsize=12)
ax.set_xticklabels(['Approved (Good Risk)', 'Rejected (High Risk)'])
```

```

for p in ax.patches:
    height = p.get_height()
    ax.annotate(f'{int(height):,}\n({height/len(df_clean)*100:.1f}%)',
                (p.get_x() + p.get_width() / 2., height / 2.0),
                ha='center', va='center', xytext=(0, 0), textcoords='offset points',
                color='white', fontweight='bold')
plt.tight_layout()
plt.savefig('static/images/approval_distribution.png', dpi=300)
plt.close()
print("Saved approval_distribution.png")

# 2. Distribution Plot of Annual Income by Approval Status (filtered to <= 500k to avoid extremely long tails)
plt.figure()
income_filtered = df_clean[df_clean['AMT_INCOME_TOTAL'] <= 500000]
ax = sns.kdeplot(data=income_filtered, x='AMT_INCOME_TOTAL', hue='label', fill=True, common_norm=False,
                 palette=['#10b981', '#ef4444'], alpha=0.5, linewidth=2)
plt.title('Annual Income Distribution by Approval Status (Income <= $500,000)', fontsize=14, fontweight='bold',
          pad=15)
plt.xlabel('Annual Income ($)', fontsize=12)
plt.ylabel('Density', fontsize=12)
# Custom legend
legend = ax.get_legend()
handles = legend.legend_handles
ax.legend(handles, ['Approved', 'Rejected'], title="Status")
plt.tight_layout()
plt.savefig('static/images/income_distribution.png', dpi=300)
plt.close()
print("Saved income_distribution.png")

# 3. Education level by Approval Status
plt.figure(figsize=(12, 7))
ax = sns.countplot(y='NAME_EDUCATION_TYPE', hue='label', data=df_clean, palette=['#10b981', '#ef4444'])
plt.title('Approval Status by Education Level', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Count', fontsize=12)
plt.ylabel('Education Level', fontsize=12)
# Custom legend
legend = ax.get_legend()
handles = legend.legend_handles
ax.legend(handles, ['Approved', 'Rejected'], title="Status")
plt.tight_layout()
plt.savefig('static/images/education_vs_approval.png', dpi=300)
plt.close()
print("Saved education_vs_approval.png")

# 4. Income Type by Approval Status
plt.figure(figsize=(12, 7))
ax = sns.countplot(y='NAME_INCOME_TYPE', hue='label', data=df_clean, palette=['#10b981', '#ef4444'])
plt.title('Approval Status by Income Type', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Count', fontsize=12)
plt.ylabel('Income Type', fontsize=12)
# Custom legend
legend = ax.get_legend()
handles = legend.legend_handles
ax.legend(handles, ['Approved', 'Rejected'], title="Status")
plt.tight_layout()
plt.savefig('static/images/income_type_vs_approval.png', dpi=300)
plt.close()
print("Saved income_type_vs_approval.png")

```

```
print("EDA generation complete successfully!")
```

3. Model Training & Evaluation Pipeline (model.py)

The file 'model.py' is responsible for training and selecting the production classifier. It addresses key developer challenges:

1. Feature Count Alignment: Restricts the training inputs to exactly the 12 features present on the Flask input form (rejecting extra columns like FLAG_MOBIL or CNT_CHILDREN) to prevent dimension mismatch crashes.
2. Categorical Pre-coding: Maps categorical strings to integers (e.g. Income Type, Housing Type, Occupation) using static, deterministic maps, aligning the training and inference values.
3. XGBoost DLL Fallback: Due to OS security policies blocking 'xgboost.dll', the pipeline falls back to scikit-learn's 'GradientBoostingClassifier' natively, ensuring the training completes successfully.
4. Target Definition: Applicants with overdue payments of 30+ days (statuses 1-5) are classified as High-Risk/Rejected (1), whereas others are classified as Approved (0).

Model Metrics Summary Table:

- Logistic Regression: Accuracy 88.23% (Predicts majority class, F1: 0.00%)
- Decision Tree Classifier: Accuracy 88.30% (F1: 39.29%)
- Random Forest Classifier (Selected): Accuracy 88.95% (F1: 42.10%)
- Gradient Boosting Classifier: Accuracy 88.25% (F1: 0.70%)

The Random Forest model has the highest classification accuracy and balance, and is saved as 'credit_model.pkl'.

Source Code: model.py

```
import pandas as pd
import numpy as np
import pickle
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Loading data...")
app = pd.read_csv('application_record.csv')
credit = pd.read_csv('credit_record.csv')

# Unique mappings (aligned with Flask forms)
gender_map = {'F': 0, 'M': 1}
car_map = {'N': 0, 'Y': 1}
realty_map = {'N': 0, 'Y': 1}
income_map = {'Working': 0, 'Commercial associate': 1, 'Pensioner': 2, 'State servant': 3, 'Student': 4}
edu_map = {'Higher education': 0, 'Secondary / secondary special': 1, 'Incomplete higher': 2, 'Lower secondary': 3, 'Academic degree': 4}
fam_map = {'Civil marriage': 0, 'Married': 1, 'Single / not married': 2, 'Separated': 3, 'Widow': 4}
housing_map = {'Rented apartment': 0, 'House / apartment': 1, 'Municipal apartment': 2, 'With parents': 3, 'Co-op apartment': 4, 'Office apartment': 5}
occ_map = {'Accountants': 0, 'Cleaning staff': 1, 'Cooking staff': 2, 'Core staff': 3, 'Drivers': 4, 'HR staff': 5, 'High skill tech staff': 6, 'IT staff': 7, 'Laborers': 8, 'Low-skill Laborers': 9, 'Managers': 10,
```

```

'Medicine staff': 11, 'Private service staff': 12, 'Realty agents': 13, 'Sales staff': 14,
'Secretaries': 15, 'Security staff': 16, 'Waiters/barmen staff': 17, 'Unknown': 18}

# Preprocessing features
print("Preprocessing features...")
app_processed = app.copy()
app_processed = app_processed.drop_duplicates(subset='ID')
app_processed['OCCUPATION_TYPE'] = app_processed['OCCUPATION_TYPE'].fillna('Unknown')

# Apply mapping
app_processed['CODE_GENDER'] = app_processed['CODE_GENDER'].map(gender_map)
app_processed['FLAG_OWN_CAR'] = app_processed['FLAG_OWN_CAR'].map(car_map)
app_processed['FLAG_OWN_REALTY'] = app_processed['FLAG_OWN_REALTY'].map(realty_map)
app_processed['NAME_INCOME_TYPE'] = app_processed['NAME_INCOME_TYPE'].map(income_map)
app_processed['NAME_EDUCATION_TYPE'] = app_processed['NAME_EDUCATION_TYPE'].map(edu_map)
app_processed['NAME_FAMILY_STATUS'] = app_processed['NAME_FAMILY_STATUS'].map(fam_map)
app_processed['NAME_HOUSING_TYPE'] = app_processed['NAME_HOUSING_TYPE'].map(housing_map)
app_processed['OCCUPATION_TYPE'] = app_processed['OCCUPATION_TYPE'].map(occ_map)

# Preprocessing target (Option B)
# C, X, 0 -> 0 (Good / Approved)
# 1-5 -> 1 (Bad / Rejected)
print("Preprocessing target labels...")
status_map = {'C': 0, 'X': 0, '0': 0, '1': 1, '2': 1, '3': 1, '4': 1, '5': 1}
credit['STATUS_NUM'] = credit['STATUS'].map(status_map)
target = credit.groupby('ID')['STATUS_NUM'].max().reset_index()
target.rename(columns={'STATUS_NUM': 'label'}, inplace=True)

# Merge features and target
df = app_processed.merge(target, on='ID', how='inner')
df = df.drop('ID', axis=1)
df = df.dropna()

# Extract exactly the 12 features that the Flask app uses
features_cols = [
    'CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'NAME_INCOME_TYPE',
    'NAME_EDUCATION_TYPE', 'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE',
    'OCCUPATION_TYPE', 'AMT_INCOME_TOTAL', 'DAYS_BIRTH', 'DAYS_EMPLOYED',
    'CNT_FAM_MEMBERS'
]
X = df[features_cols]
y = df['label']

print(f"Dataset shape: {X.shape}")
print(f"Class counts:\n{y.value_counts()}")

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Models list
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42)
}

best_model = None

```

```
best_acc = 0.0
best_name = ""

# Train and evaluate models
for name, model in models.items():
    print(f"\nTraining {name}...")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    print(f"Accuracy: {acc:.4f}")
    print("Classification Report:")
    print(classification_report(y_test, y_pred, zero_division=0))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    if acc > best_acc:
        best_acc = acc
        best_model = model
        best_name = name

print(f"\nBest Model: {best_name} (Accuracy: {best_acc:.4f})")

# Save the best performing model
print("Saving best model to credit_model.pkl...")
with open('credit_model.pkl', 'wb') as f:
    pickle.dump(best_model, f)

print("Best model trained and saved successfully!")
```


4. Web Application Backend Serving (app.py)

The server backend 'app.py' runs on Flask. It manages HTTP request routing, input parsing, feature engineering conversions, and model inference:

- Home Route ('/'): Renders the underwriting dashboard showcasing project metadata, statistics cards, and embedded Matplotlib/Seaborn visualization plots.
- Prediction Route ('/predict'): Captures the POST parameters from the input form. It converts user-entered Age in years and Employment in years into negative days (DAYS_BIRTH and DAYS_EMPLOYED) to align with the training scale. It structures the 12 features in their exact sequence, runs 'model.predict()', and renders 'result.html'.

Source Code: app.py

```
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)

# Load the best-performing credit card approval model
with open('credit_model.pkl', 'rb') as f:
    model = pickle.load(f)

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        # Extract inputs from the web form
        gender = int(request.form['gender'])
        own_car = int(request.form['own_car'])
        own_realty = int(request.form['own_realty'])
        income_type = int(request.form['income_type'])
        education = int(request.form['education'])
        family_status = int(request.form['family_status'])
        housing_type = int(request.form['housing_type'])
        occupation = int(request.form['occupation'])
        annual_income = float(request.form['annual_income'])
        age = int(request.form['age'])
        employment_years = float(request.form['employment_days'])
        family_members = int(request.form['family_members'])

        # Data Preprocessing/Scale alignment for model features:
        # Convert Age (years) to DAYS_BIRTH (negative days)
        days_birth = int(-age * 365.25)

        # Convert Employment (years) to DAYS_EMPLOYED (negative days, or 365243 for unemployed)
        if employment_years == 0:
            days_employed = 365243
        else:
            days_employed = int(-employment_years * 365.25)
```

```
# Build feature array in the exact order as training features
features = np.array([[
    gender, own_car, own_realty, income_type,
    education, family_status, housing_type,
    occupation, annual_income, days_birth,
    days_employed, family_members
]])

# Execute prediction (0 = Approved, 1 = Rejected)
prediction = model.predict(features)[0]
result = 'APPROVED' if prediction == 0 else 'REJECTED'
color = 'green' if prediction == 0 else 'red'

# Details to render in result template
details = {
    'annual_income': annual_income,
    'age': age,
    'employment': employment_years,
    'family_members': family_members,
    'gender': gender,
    'own_car': own_car,
    'own_realty': own_realty,
    'income_type': income_type
}

return render_template('result.html', result=result, color=color, details=details)

except Exception as e:
    return f'<div style="color:red; font-family:sans-serif; padding:2rem;"><h3>Error Processing
Request</h3><p>{str(e)}</p></div>'

if __name__ == '__main__':
    app.run(debug=True)
```

5. Frontend Layout Templates

The front-end user experience is managed via HTML views inside the 'templates/' folder, styled by a customized dark-mode fintech stylesheet:

- static/css/style.css: Contains the stylesheet definitions including typography (Inter), glassmorphism card filters, inputs focusing, and card-lifting hover transitions.
- templates/home.html: Features a tab-switching layout between 'Analytics & Insights' (which loads the EDA plots) and 'New Application Screening' (the submission form with matching mapped value selects).
- templates/result.html: Renders the credit decision dynamically with color-coded outcome badges (Emerald green for APPROVED, Crimson red for REJECTED) and showcases a tabular review of parameters.

Stylesheet Source Code: static/css/style.css

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

:root {
  --bg-primary: #0b0f19;
  --bg-secondary: #111827;
  --bg-card: rgba(255, 255, 255, 0.03);
  --border-color: rgba(255, 255, 255, 0.08);
  --text-primary: #f3f4f6;
  --text-secondary: #9ca3af;
  --primary-color: #6366f1;
  --primary-hover: #4f46e5;
  --success-color: #10b981;
  --error-color: #ef4444;
  --card-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.37);
  --transition-speed: 0.3s;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', sans-serif;
  background-color: var(--bg-primary);
  color: var(--text-primary);
  line-height: 1.6;
  min-height: 100vh;
  padding-bottom: 2rem;
}

/* Header & Navigation Styling */
header {
  background-color: var(--bg-secondary);
  border-bottom: 1px solid var(--border-color);
  padding: 1.5rem 2rem;
  position: sticky;
  top: 0;
  z-index: 100;
```

```
    backdrop-filter: blur(12px);
    -webkit-backdrop-filter: blur(12px);
}

.header-container {
    max-width: 1200px;
    margin: 0 auto;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.logo {
    display: flex;
    align-items: center;
    gap: 0.75rem;
    font-size: 1.5rem;
    font-weight: 700;
    color: var(--text-primary);
    text-decoration: none;
    background: linear-gradient(135deg, #a78bfa 0%, var(--primary-color) 100%);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}

.logo i {
    color: var(--primary-color);
    -webkit-text-fill-color: initial;
}

/* Layout Containers */
.main-container {
    max-width: 1200px;
    margin: 2rem auto;
    padding: 0 1.5rem;
}

/* Hero Section */
.hero {
    background: radial-gradient(circle at top right, rgba(99, 102, 241, 0.15), transparent 60%);
    border-radius: 16px;
    padding: 3rem 2rem;
    margin-bottom: 2rem;
    border: 1px solid var(--border-color);
    box-shadow: var(--card-shadow);
}

.hero h1 {
    font-size: 2.5rem;
    font-weight: 800;
    margin-bottom: 1rem;
    line-height: 1.2;
    background: linear-gradient(to right, #ffffff, #d1d5db);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}

.hero p {
```

```
font-size: 1.1rem;
color: var(--text-secondary);
max-width: 800px;
margin-bottom: 1.5rem;
}

/* Tab Navigation */
.tabs-navigation {
  display: flex;
  gap: 1rem;
  border-bottom: 1px solid var(--border-color);
  margin-bottom: 2rem;
  padding-bottom: 0.5rem;
}

.tab-btn {
  background: none;
  border: none;
  color: var(--text-secondary);
  font-size: 1.1rem;
  font-weight: 600;
  padding: 0.75rem 1.5rem;
  cursor: pointer;
  border-radius: 8px;
  transition: all var(--transition-speed) ease;
}

.tab-btn:hover {
  color: var(--text-primary);
  background-color: rgba(255, 255, 255, 0.05);
}

.tab-btn.active {
  color: var(--text-primary);
  background-color: rgba(99, 102, 241, 0.15);
  border: 1px solid rgba(99, 102, 241, 0.3);
}

.tab-content {
  display: none;
  animation: fadeIn var(--transition-speed) ease-in-out;
}

.tab-content.active {
  display: block;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

/* Stats Section */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1.5rem;
  margin-bottom: 2.5rem;
}
```

```
}

.stat-card {
  background-color: var(--bg-secondary);
  border: 1px solid var(--border-color);
  border-radius: 12px;
  padding: 1.5rem;
  text-align: center;
  box-shadow: var(--card-shadow);
  transition: transform var(--transition-speed);
}

.stat-card:hover {
  transform: translateY(-5px);
  border-color: rgba(99, 102, 241, 0.3);
}

.stat-val {
  font-size: 2rem;
  font-weight: 700;
  color: var(--primary-color);
  margin-bottom: 0.5rem;
}

.stat-label {
  font-size: 0.875rem;
  color: var(--text-secondary);
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

/* EDA Plots Gallery */
.eda-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 2rem;
  margin-bottom: 2.5rem;
}

@media (max-width: 768px) {
  .eda-grid {
    grid-template-columns: 1fr;
  }
}

.eda-card {
  background-color: var(--bg-secondary);
  border: 1px solid var(--border-color);
  border-radius: 16px;
  padding: 1.5rem;
  box-shadow: var(--card-shadow);
}

.eda-card h3 {
  font-size: 1.2rem;
  font-weight: 600;
  margin-bottom: 1rem;
  color: var(--text-primary);
}
```

```

border-left: 3px solid var(--primary-color);
padding-left: 0.75rem;
}

.eda-card img {
width: 100%;
height: auto;
border-radius: 8px;
border: 1px solid rgba(255, 255, 255, 0.05);
background-color: #ffffff; /* Contrast for charts */
padding: 0.5rem;
transition: transform var(--transition-speed);
}

.eda-card img:hover {
transform: scale(1.02);
}

/* Form Styles */
.form-card {
background-color: var(--bg-secondary);
border: 1px solid var(--border-color);
border-radius: 16px;
padding: 2.5rem;
box-shadow: var(--card-shadow);
max-width: 900px;
margin: 0 auto;
}

.form-grid {
display: grid;
grid-template-columns: repeat(2, 1fr);
gap: 1.5rem 2rem;
}

@media (max-width: 640px) {
.form-grid {
grid-template-columns: 1fr;
}
}

.form-group {
display: flex;
flex-direction: column;
gap: 0.5rem;
}

.form-group.full-width {
grid-column: span 2;
}

@media (max-width: 640px) {
.form-group.full-width {
grid-column: span 1;
}
}

label {

```

```
font-size: 0.9rem;
font-weight: 500;
color: var(--text-primary);
}

input[type="number"], select {
  background-color: rgba(255, 255, 255, 0.04);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 0.8rem 1rem;
  color: var(--text-primary);
  font-size: 1rem;
  font-family: inherit;
  transition: all var(--transition-speed);
  outline: none;
  width: 100%;
}

input[type="number"]::placeholder {
  color: rgba(255, 255, 255, 0.2);
}

input[type="number"]:focus, select:focus {
  border-color: var(--primary-color);
  background-color: rgba(255, 255, 255, 0.08);
  box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.25);
}

select option {
  background-color: var(--bg-secondary);
  color: var(--text-primary);
}

.btn-container {
  display: flex;
  justify-content: center;
  margin-top: 2rem;
}

.submit-btn {
  background: linear-gradient(135deg, var(--primary-color) 0%, var(--primary-hover) 100%);
  color: white;
  font-size: 1.1rem;
  font-weight: 600;
  border: none;
  border-radius: 8px;
  padding: 1rem 3rem;
  cursor: pointer;
  transition: all var(--transition-speed);
  box-shadow: 0 4px 12px rgba(99, 102, 241, 0.3);
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.submit-btn:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 20px rgba(99, 102, 241, 0.5);
}
```



```
background: linear-gradient(135deg, #7c3aed 0%, var(--primary-color) 100%);
}

.submit-btn:active {
  transform: translateY(0);
}

/* Result Dashboard Styling */
.result-card {
  background-color: var(--bg-secondary);
  border: 1px solid var(--border-color);
  border-radius: 20px;
  padding: 3rem;
  box-shadow: var(--card-shadow);
  max-width: 600px;
  margin: 3rem auto;
  text-align: center;
  position: relative;
  overflow: hidden;
}

.result-card::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 6px;
}

.result-card.approved::before {
  background: var(--success-color);
}

.result-card.rejected::before {
  background: var(--error-color);
}

.result-icon {
  font-size: 4.5rem;
  margin-bottom: 1.5rem;
}

.result-badge {
  display: inline-block;
  padding: 0.5rem 1.5rem;
  border-radius: 50px;
  font-weight: 700;
  font-size: 1.25rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  margin-bottom: 1.5rem;
}

.result-badge.approved {
  background-color: rgba(16, 185, 129, 0.15);
  color: var(--success-color);
  border: 1px solid rgba(16, 185, 129, 0.3);
}
```

```
}

.result-badge.rejected {
  background-color: rgba(239, 68, 68, 0.15);
  color: var(--error-color);
  border: 1px solid rgba(239, 68, 68, 0.3);
}

.result-message {
  color: var(--text-secondary);
  font-size: 1.1rem;
  margin-bottom: 2.5rem;
}

.back-btn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  background: transparent;
  border: 1px solid var(--border-color);
  color: var(--text-primary);
  text-decoration: none;
  font-weight: 500;
  padding: 0.8rem 2rem;
  border-radius: 8px;
  transition: all var(--transition-speed);
}

.back-btn:hover {
  background-color: rgba(255, 255, 255, 0.05);
  border-color: var(--text-secondary);
  transform: translateX(-3px);
}

/* Footer Styling */
footer {
  text-align: center;
  color: var(--text-secondary);
  font-size: 0.875rem;
  margin-top: 4rem;
  padding-top: 2rem;
  border-top: 1px solid var(--border-color);
}
```

Template Source Code: templates/home.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Credit Card Approval System - Intelligent Screening Dashboard</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css">
</head>
<body>

  <header>
    <div class="header-container">
      <a href="/" class="logo">
        <i class="fa-solid fa-credit-card"></i>
        <span>CreditCardAI</span>
      </a>
      <div style="font-size: 0.9rem; color: var(--text-secondary);">
        <i class="fa-solid fa-shield-halved"></i> Secured Analytics Platform
      </div>
    </div>
  </header>

  <div class="main-container">
    <!-- Hero Section -->
    <section class="hero">
      <h1>Automated Credit Card Decisioning System</h1>
      <p>
        An intelligent underwriting platform that automates credit card decisions using advanced
        machine learning models trained on historical customer financial profiles and credit records.
      </p>
      <div style="display: flex; gap: 1rem; flex-wrap: wrap;">
        <span class="back-btn" style="padding: 0.4rem 1rem; font-size: 0.85rem; border-color:
        rgba(99,102,241,0.3); pointer-events: none;">
          <i class="fa-solid fa-bolt"></i> Real-time Evaluation
        </span>
        <span class="back-btn" style="padding: 0.4rem 1rem; font-size: 0.85rem; border-color:
        rgba(16,185,129,0.3); pointer-events: none; color: var(--success-color);">
          <i class="fa-solid fa-circle-check"></i> Random Forest (Acc: 88.95%)
        </span>
      </div>
    </section>

    <!-- Tab Controls -->
    <div class="tabs-navigation">
      <button class="tab-btn active" onclick="switchTab('analytics')">
        <i class="fa-solid fa-chart-line"></i> Analytics & Insights
      </button>
      <button class="tab-btn" onclick="switchTab('predict')">
        <i class="fa-solid fa-user-plus"></i> New Application Screening
      </button>
    </div>

    <!-- Tab 1: Analytics -->
    <div id="analytics" class="tab-content active">

```

```

<!-- Model Performance Metrics Card -->
<div class="stats-grid">
  <div class="stat-card">
    <div class="stat-val">88.95%</div>
    <div class="stat-label">Model Accuracy</div>
  </div>
  <div class="stat-card">
    <div class="stat-val">36,457</div>
    <div class="stat-label">Records Processed</div>
  </div>
  <div class="stat-card">
    <div class="stat-val">0.94</div>
    <div class="stat-label">Good Client F1-Score</div>
  </div>
  <div class="stat-card">
    <div class="stat-val">4 Algorithms</div>
    <div class="stat-label">Evaluated</div>
  </div>
</div>

<!-- EDA Visualization Plots -->
  <h2 style="margin-bottom: 1.5rem; border-bottom: 1px solid var(--border-color); padding-bottom: 0.5rem;">Exploratory Data Analysis Insights</h2>
  <div class="eda-grid">
    <div class="eda-card">
      <h3>Approval Distribution</h3>
      
        Proportion of credit applications flagged as Good (Approved) vs High-Risk/Delinquent (Rejected) based on historical monthly status.
      </p>
    </div>
    <div class="eda-card">
      <h3>Income Density by Approval</h3>
      
        Kernel density estimate showing total annual income distribution (truncated at $500k) separated by credit decision outcomes.
      </p>
    </div>
    <div class="eda-card">
      <h3>Education Impact</h3>
      
        Segmented count plot representing credit application outcomes across various academic qualifications.
      </p>
    </div>
    <div class="eda-card">
      <h3>Income Category Impact</h3>
      
        Distribution of approvals/rejections across different types of income sources (Working, Commercial, State servant, Pensioner, Student).
      </p>
    </div>
  </div>

```

```

        </p>
      </div>
    </div>
  </div>

  <!-- Tab 2: Predict Form -->
  <div id="predict" class="tab-content">
    <div class="form-card">
      <h2 style="margin-bottom: 1.5rem; text-align: center;">Applicant Eligibility
Evaluation</h2>
      <p style="color: var(--text-secondary); font-size: 0.95rem; text-align: center;
margin-bottom: 2.5rem; max-width: 600px; margin-left: auto; margin-right: auto;">
        Provide applicant's demographic profile, financial records, and employment duration
to compute credit approval recommendation instantly.
      </p>

      <form action="/predict" method="POST">
        <div class="form-grid">

          <!-- Gender -->
          <div class="form-group">
            <label for="gender">Applicant Gender</label>
            <select name="gender" id="gender" required>
              <option value="" disabled selected>Select gender</option>
              <option value="0">Female</option>
              <option value="1">Male</option>
            </select>
          </div>

          <!-- Own Car -->
          <div class="form-group">
            <label for="own_car">Owns a Car?</label>
            <select name="own_car" id="own_car" required>
              <option value="" disabled selected>Select option</option>
              <option value="1">Yes</option>
              <option value="0">No</option>
            </select>
          </div>

          <!-- Own Realty -->
          <div class="form-group">
            <label for="own_realty">Owns Real Estate / Realty?</label>
            <select name="own_realty" id="own_realty" required>
              <option value="" disabled selected>Select option</option>
              <option value="1">Yes</option>
              <option value="0">No</option>
            </select>
          </div>

          <!-- Income Type -->
          <div class="form-group">
            <label for="income_type">Income Source Type</label>
            <select name="income_type" id="income_type" required>
              <option value="" disabled selected>Select source</option>
              <option value="0">Working</option>
              <option value="1">Commercial Associate</option>
              <option value="2">Pensioner</option>
              <option value="3">State Servant</option>

```

```

        <option value="4">Student</option>
    </select>
</div>

<!-- Education Level -->
<div class="form-group">
    <label for="education">Highest Education Level</label>
    <select name="education" id="education" required>
        <option value="" disabled selected>Select education</option>
        <option value="0">Higher Education</option>
        <option value="1">Secondary / Secondary Special</option>
        <option value="2">Incomplete Higher</option>
        <option value="3">Lower Secondary</option>
        <option value="4">Academic Degree</option>
    </select>
</div>

<!-- Family Status -->
<div class="form-group">
    <label for="family_status">Marital / Family Status</label>
    <select name="family_status" id="family_status" required>
        <option value="" disabled selected>Select family status</option>
        <option value="0">Civil Marriage</option>
        <option value="1">Married</option>
        <option value="2">Single / Not Married</option>
        <option value="3">Separated</option>
        <option value="4">Widow</option>
    </select>
</div>

<!-- Housing Type -->
<div class="form-group">
    <label for="housing_type">Current Housing Situation</label>
    <select name="housing_type" id="housing_type" required>
        <option value="" disabled selected>Select housing</option>
        <option value="1">House / Apartment</option>
        <option value="0">Rented Apartment</option>
        <option value="2">Municipal Apartment</option>
        <option value="3">With Parents</option>
        <option value="4">Co-op Apartment</option>
        <option value="5">Office Apartment</option>
    </select>
</div>

<!-- Occupation Type -->
<div class="form-group">
    <label for="occupation">Occupation / Job Sector</label>
    <select name="occupation" id="occupation" required>
        <option value="" disabled selected>Select occupation</option>
        <option value="8">Laborers</option>
        <option value="14">Sales Staff</option>
        <option value="3">Core Staff</option>
        <option value="10">Managers</option>
        <option value="4">Drivers</option>
        <option value="0">Accountants</option>
        <option value="11">Medicine Staff</option>
        <option value="6">High Skill Tech Staff</option>
        <option value="16">Security Staff</option>
    </select>
</div>

```

```

        <option value="2">Cooking Staff</option>
        <option value="1">Cleaning Staff</option>
        <option value="12">Private Service Staff</option>
        <option value="9">Low-Skill Laborers</option>
        <option value="15">Secretaries</option>
        <option value="17">Waiters / Barmen Staff</option>
        <option value="5">HR Staff</option>
        <option value="7">IT Staff</option>
        <option value="13">Realty Agents</option>
        <option value="18">Other / Unemployed / Unknown</option>
    </select>
</div>

<!-- Annual Income -->
<div class="form-group">
    <label for="annual_income">Total Annual Income ($)</label>
    <input type="number" step="100" min="5000" max="10000000"
name="annual_income" id="annual_income" placeholder="e.g. 55000" required>
</div>

<!-- Age -->
<div class="form-group">
    <label for="age">Age (Years)</label>
    <input type="number" min="18" max="100" name="age" id="age"
placeholder="e.g. 34" required>
</div>

<!-- Employment Duration -->
<div class="form-group">
    <label for="employment_days">Employment Duration (Years)</label>
    <input type="number" min="0" max="60" name="employment_days"
id="employment_days" placeholder="e.g. 8 (Enter 0 if Unemployed)" required>
</div>

<!-- Family Members -->
<div class="form-group">
    <label for="family_members">Number of Family Members</label>
    <input type="number" min="1" max="20" name="family_members"
id="family_members" placeholder="e.g. 3" required>
</div>

</div>

<div class="btn-container">
    <button type="submit" class="submit-btn">
        <i class="fa-solid fa-magnifying-glass"></i> Evaluate Eligibility
    </button>
</div>
</form>
</div>
</div>
</div>

<footer>
    <p>&copy; 2026 Credit Card Approval Prediction Project. Powered by Scikit-Learn & Flask.</p>
</footer>

<script>

```

```
function switchTab(tabId) {
  // Remove active classes
  document.querySelectorAll('.tab-btn').forEach(btn => btn.classList.remove('active'));
  document.querySelectorAll('.tab-content').forEach(content =>
content.classList.remove('active'));

  // Add active classes
  const activeBtn = Array.from(document.querySelectorAll('.tab-btn')).find(btn =>
btn.onclick.toString().includes(tabId));
  if (activeBtn) activeBtn.classList.add('active');

  const targetContent = document.getElementById(tabId);
  if (targetContent) targetContent.classList.add('active');
}
</script>
</body>
</html>
```


Template Source Code: templates/result.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Evaluation Result - Credit Card Application</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
</head>
<body>

  <header>
    <div class="header-container">
      <a href="/" class="logo">
        <i class="fa-solid fa-credit-card"></i>
        <span>CreditCardAI</span>
      </a>
      <div style="font-size: 0.9rem; color: var(--text-secondary);">
        <i class="fa-solid fa-shield-halved"></i> Underwriting Panel
      </div>
    </div>
  </header>

  <div class="main-container">
    <div class="result-card {{ 'approved' if color == 'green' else 'rejected' }}">
      {% if color == 'green' %}
        <div class="result-icon" style="color: var(--success-color);">
          <i class="fa-solid fa-circle-check"></i>
        </div>
        <div class="result-badge approved">APPROVED</div>
        <p class="result-message">
          Based on the applicant's demographic profile, income scale, and stability metrics,
          the model recommends <strong>immediate card approval</strong>.
        </p>
      {% else %}
        <div class="result-icon" style="color: var(--error-color);">
          <i class="fa-solid fa-circle-xmark"></i>
        </div>
        <div class="result-badge rejected">REJECTED</div>
        <p class="result-message">
          The evaluation indicates a high credit risk profile due to income, stability or debt
          metrics. The application has been flagged for <strong>further manual review</strong>.
        </p>
      {% endif %}

      <!-- Submitted Details Summary -->
      <div style="background-color: rgba(255,255,255,0.02); border: 1px solid var(--border-color);
border-radius: 8px; padding: 1.5rem; text-align: left; margin-bottom: 2rem;">
        <h4 style="margin-bottom: 1rem; color: var(--text-primary); border-bottom: 1px solid
var(--border-color); padding-bottom: 0.5rem; font-size: 1rem;">
          <i class="fa-solid fa-list-check"></i> Profile Parameters Analyzed
        </h4>
        <ul style="list-style-type: none; display: grid; grid-template-columns: 1fr 1fr; gap:
0.75rem 1.5rem; font-size: 0.875rem;">
          <li><span style="color: var(--text-secondary);">Annual Income:</span> <strong>${{

```

```

"{:, .2f}".format(details.annual_income) }}</strong></li>
    <li><span style="color: var(--text-secondary);">Age:</span> <strong>{{ details.age
}} Years</strong></li>
    <li><span style="color: var(--text-secondary);">Employment:</span> <strong>{{
details.employment }} Years</strong></li>
    <li><span style="color: var(--text-secondary);">Family Size:</span> <strong>{{
details.family_members }} Members</strong></li>
    <li><span style="color: var(--text-secondary);">Gender:</span> <strong>{{ 'Male' if
details.gender == 1 else 'Female' }}</strong></li>
    <li><span style="color: var(--text-secondary);">Car Owner:</span> <strong>{{ 'Yes'
if details.own_car == 1 else 'No' }}</strong></li>
    <li><span style="color: var(--text-secondary);">Realty Owner:</span> <strong>{{
'Yes' if details.own_realty == 1 else 'No' }}</strong></li>
    <li><span style="color: var(--text-secondary);">Income Category:</span>
    <strong>
        {% if details.income_type == 0 %}Working
        {% elif details.income_type == 1 %}Commercial associate
        {% elif details.income_type == 2 %}Pensioner
        {% elif details.income_type == 3 %}State servant
        {% elif details.income_type == 4 %}Student
        {% else %}Other{% endif %}
    </strong>
    </li>
</ul>
</div>

<a href="/" class="back-btn">
    <i class="fa-solid fa-arrow-left"></i> Run Another Screening
</a>
</div>
</div>

<footer>
    <p>&copy; 2026 Credit Card Approval Prediction Project. Powered by Scikit-Learn & Flask.</p>
</footer>

</body>
</html>

```

6. Command-Line Reference Guide

To execute and host this system locally, execute the following commands in order:

1. Open PowerShell or Command Prompt in the project workspace directory:

```
cd c:\Users\mah9\Desktop\creditcard_approval_project
```

2. Activate the python virtual environment (venv):

In PowerShell: `.\venv\Scripts\Activate.ps1`

In CMD: `.\venv\Scripts\activate.bat`

3. Run exploratory charts generation:

```
python eda.py
```

4. Train all classifiers, print comparative reports, and export Random Forest:

```
python model.py
```

5. Run the web server:

```
python app.py
```

6. In your web browser, navigate to: `http://127.0.0.1:5000/`